

AGH

AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI, INFORMATYKI I INŻYNIERII BIOMEDYCZNEJ

Praca dyplomowa

*Design, Construction, and Control of a 6-DoF Robotic Arm
Manufactured Using 3D Printing*

*Projektowanie, budowa i sterowanie sześćcioosiowym
ramieniem robota wykonanym w technologii druku 3D*

Autor:

inż. Adrian Paś

Kierunek studiów:

Automatyka i Robotyka

Opiekun pracy:

dr hab. inż. Adam Piłat, prof. u.

Kraków, 2026

Contents

1	Introduction	5
1.1	Aim of this thesis	5
1.2	Scope of this thesis	5
2	Literature review and existing solutions	6
2.1	Robotic manipulators - classification and application	6
2.2	Different ways of constructing a robotic arm	7
2.3	Existing projects of 6-DoF manipulators	7
3	Project assumptions and constraints	9
3.1	Mechanical constraints	9
3.2	Operational parameters	9
3.3	Electronic constraints	9
3.4	Safety constraints	10
4	Mechanical design	11
4.1	CAD model of the manipulator	11
4.2	Mechanical specifications and dimensions	12
4.3	Calculating torque for two main joints	13
4.4	Optimalization for 3D printing	15
5	Actuation and electronics	17
5.1	Motors with gears	17
5.2	Positioning and feedback	18
5.3	Control unit - uC, drivers	18
5.4	Electronic diagram	19
6	Mathematical model	20
6.1	Forward kinematics	20
6.2	Inverse kinematics	22
6.2.1	Position of the wrist	22
6.2.2	Orientation of the wrist	24
6.3	Summary of inverse kinematics and constraints	26
6.4	Algorithms of TCP control	27
6.4.1	Path linearization and interpolation	27
6.4.2	Joint synchronization	27
6.4.3	Look-ahead and velocity planning	28

7	Software	29
7.1	Programming languages used	29
7.1.1	Python	29
7.1.2	C	29
7.2	Communication computer - robot	29
7.3	Graphical User Interface (GUI)	30
7.4	Simulation and IK debugging	32
8	Validation and testing	33
8.1	Load tests	33
8.1.1	Shoulder joint (joint 2) evaluation	33
8.1.2	Elbow joint (joint 3) evaluation	35
8.2	TCP trajectory test	36
8.2.1	Linear trajectory	36
8.2.2	Semicircular trajectory	38
8.2.3	Trajectory Exceptions	40
9	Conclusions	41
9.1	Summary of results	41
9.2	Evaluation of potential usage in real life applicaiton	41
9.3	Future improvements	42

1 Introduction

1.1 Aim of this thesis

In modern factories, automation and robotization have become crucial in improving efficiency, precision, and safety. Industrial manipulators are increasingly used to replace or assist humans in tedious or hazardous tasks, such as palletizing, assembly, and conveyor belt operations. This generates a need for highly qualified engineers with the skills to design, program, and maintain robotic systems.

However, training future engineers on industrial robots is not practical due to the high cost of manipulators, costly fixing and maintenance, and the safety related to operating a machine with such a high payload. There is a clear need for affordable, safe, and easy to fix robots that mimic the kinematics and control used in industrial standards.

The main aim of this thesis is to design, construct, and evaluate a fully functional 6 DoF articulated robotic arm intended for educational purposes. The primary objective was to create a platform that mimics the configuration and control logic of industrial manipulators, while remaining safe for human interaction, affordable, and easy to maintain. Using 3D printing technology and combining an STM32 microcontroller with a Python control system, this project provides a perfect tool to study advanced robotics, such as inverse kinematics, joint synchronization, and path planning.

1.2 Scope of this thesis

This thesis is structured into several chapters that describe the entire process of designing, constructing, and programming the robotic arm, with functional testing of the physical platform.

Following this introduction, Chapter 2 provides examples of existing robotic arms and similar open source projects. Chapter 3 defines the project assumptions, such as mechanical, electronic and safety constraints. In Chapters 4 and 5, these constraints are implemented, starting with mechanical design, CAD modeling, and torque calculations, to the selection of actuators, electronics, and the main control unit. Chapter 6 focuses on the mathematical model of the manipulator and explains forward and inverse kinematics with path planning and joint synchronization algorithms. Chapter 7 presents the software used, communication protocols, and the graphical user interface with the simulation tools. Chapter 8 evaluates the performance of the completed arm with load tests and trajectory verification. Finally, Chapter 9 provides a conclusion, summarizes the experimental results, and discusses potential real-life applications and future improvements for the system.

2 Literature review and existing solutions

The topic of robotic manipulators dates back to 1954, when the first hydraulic arm was invented and constructed in the United States.[1] Since then, many improvements have been made. Today, not only are there many types of robotic manipulators, but also most of them are equipped with numerous sensors and cameras.

2.1 Robotic manipulators - classification and application

Robotic arms can be divided into the classification presented in Fig. 1. Each of the constructions has its own advantages and disadvantages.

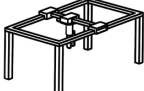
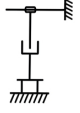
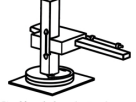
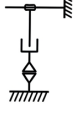
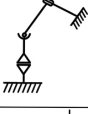
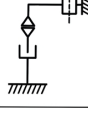
Principle	Kinematic Structure	Workspace
 Cartesian Robot		
 Cylindrical Robot		
 Spherical Robot		
 SCARA Robot		
 Articulated Robot		

Figure 1. Classification of robotics manipulators [2].

The first group consists of cartesian manipulators, which are the simplest to control. The main drawback is the platform that has a large volume, usually greater than the robot's workspace. The best example of this mechanism is an overhead crane.

The second category are robots that stand on their own on a rotating base. This type is the most common in industrial applications. For example, SCARA robots are commonly used by conveyor belts or whenever parts need to be moved with high precision. For other, general purpose applications the articulated robots are almost always used. They have all of their joints as rotational, which allows for the greatest flexibility. This is the type of robot that is presented in this thesis. They can be found in assembly lines for cars, welding applications, or palletizing. That is why there are numerous brands that sell this type of robot. The most popular are Fanuc, Kuka, or ABB.

2.2 Different ways of constructing a robotic arm

Robotic manipulators for industrial applications are generally made with metal as they need to be rigid to withstand huge loads. Because of that, their construction has a number of constraints. They are extremely heavy, require huge motors, consume a lot of power, and are dangerous to people standing in their operating range. All of those issues make it impractical for any other application than industrial applications.

Another way of making robotic arms is 3D printing. This technique allows for the creation of safer robots. Those plastic versions, although less accurate, repeatable, or rigid, are the exact copy of the metal manipulators when it comes to construction. That makes them the perfect candidates for use in schools as a training tool for future engineers. In addition, if something goes wrong, making spare parts and fixing the 3d printed manipulator takes much less time and money.

2.3 Existing projects of 6-DoF manipulators

As robotic arms are extremely common in industrial applications, there is a great deal of them, ranging from big companies to hobbistic models. The following are some examples of the most popular ones.



Figure 2. Fanuc M-1000iA [3].



Figure 3. Educational robot Astorino [4].



Figure 4. AR4 hobbyist educational robot [5].

3 Project assumptions and constraints

The main goal of this project was to design a manipulator that closely follows the architecture of industrial robotic arms while remaining safe for human interaction. To ensure that the robot is functional and reliable, several mechanical, operational, electronic, and safety constraints had to be defined.

3.1 Mechanical constraints

A key assumption was to implement a 6-degree-of-freedom (6-DOF) structure. This configuration mimics industrial standards, where the first three joints position the wrist in space and the remaining three determine the orientation of the end effector. This setup ensures that the control logic and kinematics of the robot are kept similar to those of the industrial manipulators.

Material selection was another important constraint. Since the robot is mostly 3D-printed, it was necessary to choose a filament that balances ease of printing with structural integrity. The chosen material had to provide enough precision for the gear teeth and bearings while being fast enough to print larger components. The final design aims to be rigid enough to prevent flexing under its own weight, yet light enough to be powered by hobbyist-grade motors.

3.2 Operational parameters

The operational group of assumptions involves the robot's physical capabilities, such as reach and motion limits

- **Reach:** The maximum reach was determined by the 3D printer's build plate size. Arm links were designed to be as large as the printer could handle.
- **Motion Limits:** Each joint has a defined range of motion for two reasons. First, it prevents cables from tangling or breaking during rotation. Second, it allows the use of limit switches for the homing procedure, which establishes a "zero" position in space.
- **Load Capacity:** High strength was not a priority for this project. Since the structure is plastic, heavy loads could cause the links to bend or crack over time. The robot is intended for light tasks, such as moving small objects or playing board games, where the load is negligible.

3.3 Electronic constraints

Electronic constraints are just as important as mechanical ones and can be divided into three categories: the main control unit, the motors, and the power supply.

The main constraint in selecting the control unit was computational performance. For a six degree of freedom manipulator, the kinematics equations are quite heavy, so a board with a high speed microprocessor and floating point support was necessary. Another factor was the number of inputs and outputs. Since the project uses many motors, limit switches, and sensors, the board must handle all of them simultaneously without any lag.

The second group of components are the motors. Here, torque, weight, cost, and power consumption needed to be balanced. A compromise had to be established between all these parameters. For the base axis, which holds the weight of the entire arm, torque was the priority over weight and cost. However, for the joints further up the arm, weight became the most important factor, the lighter the motor, the better. The overall cost was kept low since this is a hobbyist project made of 3D printed plastic parts, and using expensive industrial motors would be overkill.

The last thing to consider was the power supply. The main power consumption comes from the motors. It was assumed that the operating voltage should be maximized within the limits of the drivers. This will ensure that the motors maintain a high level of torque, even at higher speeds.

3.4 Safety constraints

The safety constraints in this project are mainly focused on the control and motion of the robotic arm. Unlike heavy industrial robots made of steel, which can pose a significant danger to operators, the manipulator built in this thesis is 3D printed and operates at lower power levels. This design results in a much lower risk for humans as the motors do not provide enough force to cause serious injury.

The main safety concerns are actually related to the robot itself, as collisions with other objects could easily damage the manipulator. To prevent this, software limitations were implemented. The first was the virtual boundarie to keep the end effector from hitting the ground and other known obstacles. The second was to limit each joint to make it work within a safe operating range so that the robot would not tangle its cables or hit itself.

4 Mechanical design

The robotic arm was fully designed and built by the author of this thesis. The design is based on standard robotic manipulators, but the main goal was to see how many parts could be 3D printed. Using 3D printing makes the robot much easier to repair because there is no need to wait for expensive or specialized parts from a shop. If something breaks, a new part can be printed.

The entire project was designed in FreeCAD. This program was chosen because it is free and works locally on the computer, so it does not require a cloud connection or a user account. Additionally, using this software means the author has full control and ownership of all the designs without any licensing problems.

4.1 CAD model of the manipulator

The following figure shows the complete 3D model of the robotic arm (without the gripper), with a list of parts on the left.

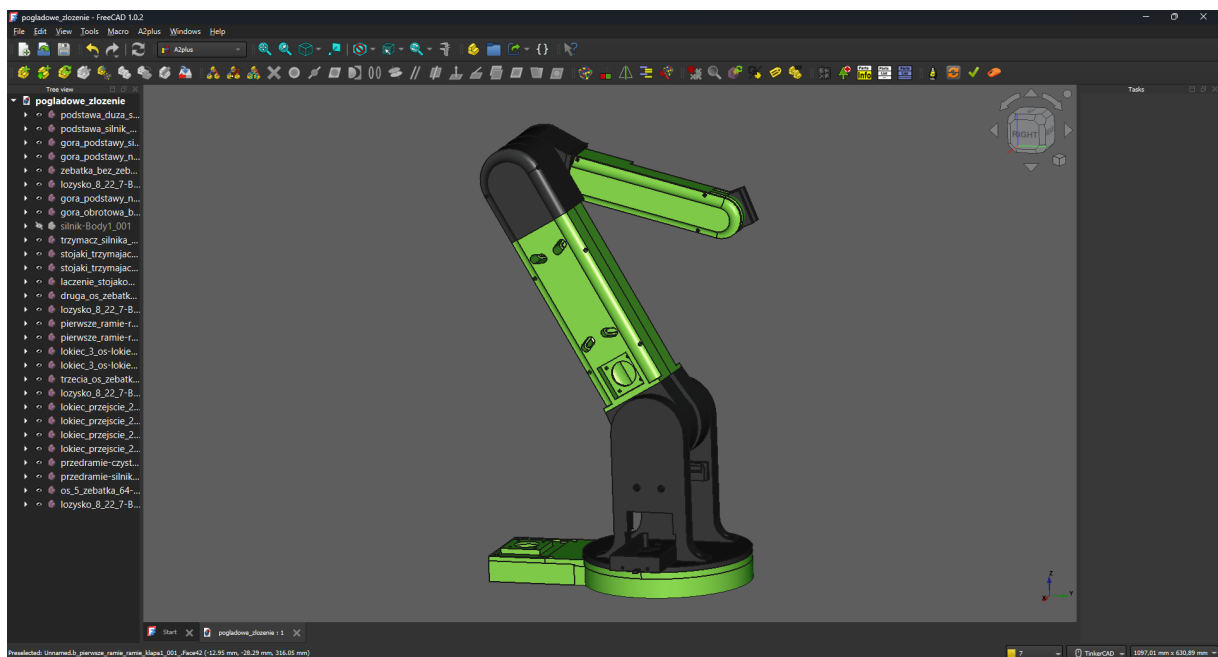


Figure 5. Complete model of the robotic arm in FreeCAD. Source: Own work

The manipulator consists of 24 3D-printed parts, including:

- gears with 32, 64, and 128 teeth,
- custom-designed "lazy susan" ball bearings,
- timing belt tensioners.

Specialized bearings were designed for the main base turntable, the elbow, and the forearm. This helped reduce the overall weight of the robot and, more importantly, allowed custom bearing sizes that perfectly fit the design. These bearings use 5mm metal balls and a 3D-printed race with even spacing to keep the balls in place while allowing them to spin freely. As mentioned

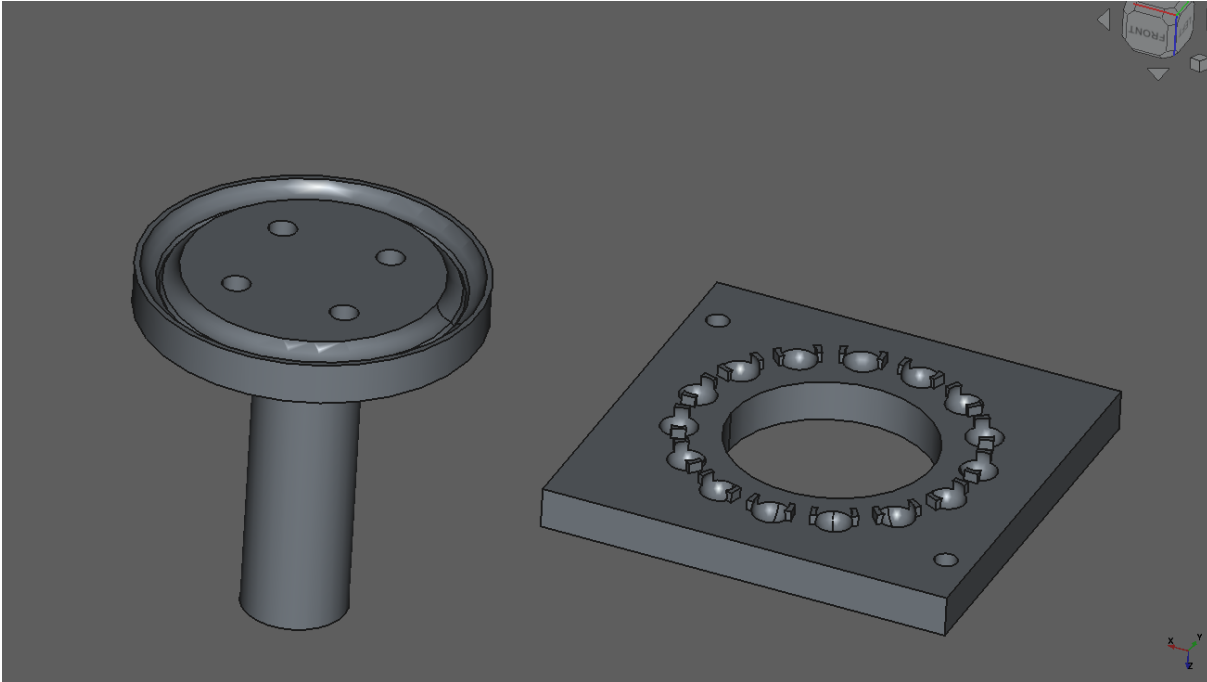


Figure 6. Custom ball bearing design. Source: Own work

above, the main goal was to design a functional robotic arm using as few standard parts as possible. In addition to electronics, the only components that need to be purchased are threaded inserts, screws, small standard bearings, timing belts, and metal balls.

Although these parts must be purchased, they are all standard, widely available products. They are not specific to this single project, so they are always in stock and easy to find. This fulfills the main goal of the thesis: creating a robotic arm primarily from 3D-printed materials without being dependent on specialized suppliers.

4.2 Mechanical specifications and dimensions

After completing the CAD model, the physical limits and dimensions of the manipulator were established. Defining these parameters is essential for understanding the robot's workspace and for future calculations in the control software.

The reach of the robot is determined by the lengths of the individual links measured between the rotation axes of the joints.

- **Base height (Link 0):** 187.3 mm – from the ground to the center of the shoulder joint.

- **Upper arm (Link 1):** 335.6 mm – from the shoulder to the elbow joint.
- **Forearm (Link 2):** 228.6 mm - from the elbow to the center of the wrist.
- **Wrist to TCP (Link 3):** 212 mm – from the center of the wrist to the end of the gripper.

To provide a quick overview of the robot's capabilities, the most important technical specifications are presented in the table below.

Table 1. General technical specifications of the manipulator.

Parameter	Value
Architecture	6-Axis Articulated Robot
Maximum Reach (Radius)	776.2 mm
Total Weight	approx. 4.4 kg
Main Material	PLA (3D Printed)
Drive System	NEMA 17 and 23 Stepper Motors
Transmission	GT2 Timing Belts / 3D Printed Gears
Estimated Payload	Up to 500 g with 1/16 microsteps
Base Diameter	220 mm

4.3 Calculating torque for two main joints

To ensure that the stepper motors were capable of holding the manipulator, it was necessary to calculate the required torque for the two main joints, the second and third, which are the most stressed during the operations. The most demanding position is when the robotic arm is fully extended horizontally.

The torque for each individual part was calculated using the following formula:

$$M = m \cdot g \cdot r \quad (1)$$

where m is the mass of the component in kg, g is the gravitational acceleration (9.81 m/s^2), and r is the distance from the center of mass of the part to the axis of the joint in meters.

The mass of each 3D-printed part was obtained from the Bambu Studio slicer, while the mass of the motors was taken from their datasheets [6]. For these calculations, it was assumed that the center of gravity for each part lies at its geometric center. The weight of minor components such as screws, nuts, and timing belts was considered negligible.

The table below presents the torque calculation for the second joint (Shoulder), which uses a NEMA 23 motor.

Table 2. Torque calculation for Nema23 (Axis 2)

Item	Weight [kg]	Center of Gravity [m]	Torque [Nm]
nema17 + bearing	0.289	0.070	0.201
arm bearing	0.012	0.179	0.021
arm	0.210	0.154	0.324
arm gear	0.014	0.179	0.025
elbow	0.089	0.319	0.284
elbow gear	0.041	0.334	0.137
elbow bearing	0.012	0.334	0.038
entire elbow	0.084	0.334	0.282
nema17 + bearing (2)	0.289	0.294	0.849
small motor	0.132	0.405	0.535
forearm	0.104	0.480	0.497
axis 5 bearings	0.012	0.567	0.065
axis 5 gear	0.014	0.567	0.081
axis 6 holder	0.015	0.605	0.088
servo sg995	0.055	0.620	0.341
pointer	0.011	0.711	0.080
Total Sum			3.847

The total torque of 3.85 Nm is higher than the maximum holding torque for the chosen Nema23 motor (57HS112-3004), which is around 3 Nm in the datasheet [7]. To overcome this, a gear ratio of 128:20 (6.4:1) was applied on this axis.

The same was done for the third joint (Elbow).

Table 3. Torque calculation for Nema17 (Axis 3)

Item	Weight [kg]	Center of Gravity [m]	Torque [Nm]
nema17 + bearing	0.289	-0.040	-0.115
small motor	0.132	0.071	0.092
forearm	0.104	0.146	0.148
axis 5 bearings	0.012	0.233	0.026
axis 5 gear	0.014	0.233	0.033
axis 6 holder	0.015	0.271	0.039
servo sg995	0.055	0.286	0.154
pointer	0.011	0.377	0.042
Total Sum			0.419

Here again, the total torque exceeds the maximum holding torque for the motor used Nema17 (HS4401), which is 0.4 Nm according to the datasheet [6]. To overcome this, a gear ratio of 8:1 was applied on this axis.

4.4 Optimalization for 3D printing

Optimizing the design for 3D printing is an essential part of the process. Several constraints and techniques must be considered during the design phase.

- The size of the 3D printer build plate.
- The printer's precision and layer height.
- Wall thickness and infill percentage.
- Printing time, while not the most critical factor, printing smaller parts reduces the risk of total failure during long jobs.

The printer used for this project was a Bambu Lab A1. It has a build volume of $256 \times 256 \times 256 \text{ mm}^3$. All parts were printed using a standard 0.4 mm nozzle, as it provides a good balance between printing speed and detail. The layer height was set to 0.2 mm. To ensure that parts are strong enough but still lightweight, most structural elements were designed with a total thickness of 5 mm. This thickness consists of 3 layers of the outer wall on each side, with a 15% infill between. This combination provides great rigidity while keeping the weight of the robotic arm low.

In addition to these settings, certain design techniques can significantly improve the quality of the robot.

- Proper orientation of the object on the build plate.
- Using supports only where necessary.
- Splitting large designs into smaller, connected parts.

In Fused Deposition Modeling (FDM) in 3D printing, the internal bond of the filament is almost always stronger than the adhesive bond between the printed layers [8]. Because of this, the printing orientation was the most important parameter. Choosing the right orientation was always a priority, even if it required more supports or a longer printing time.

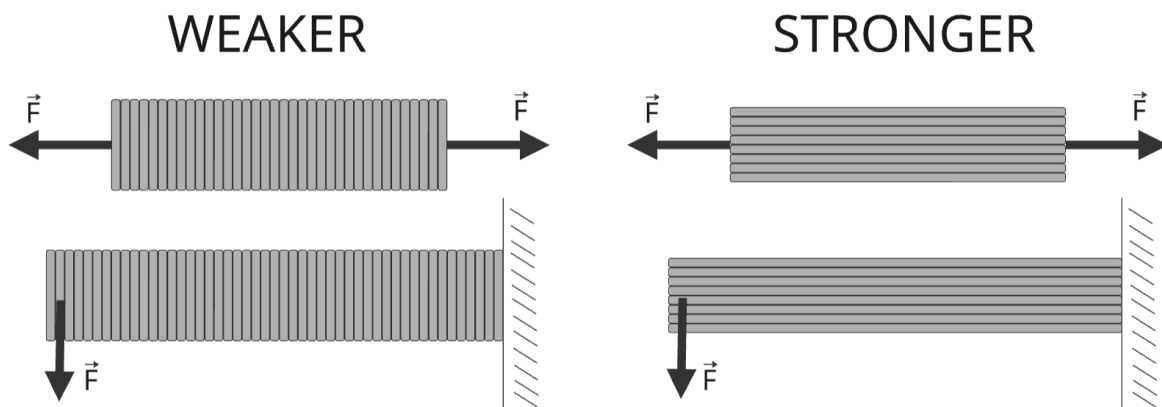


Figure 7. Comparison of strength based on printing orientation. Source: Own work

Another important factor is deciding when to use supports. A good way to optimize the design is to eliminate the need for supports by changing the geometry. For example, a 90-degree overhang can often be replaced with a slope (chamfer) to make it self-supporting. This saves material and avoids the "ragged" surface finish that supports often leave behind.

The final technique used was to split the design into smaller components. Although it is usually easier to print one large part, splitting is useful in two cases. First, it allows for better access to the inside of a part to install gears or cables. Second, it makes assembly easier by providing more space for tools, such as screwdrivers, during the final build.

5 Actuation and electronics

5.1 Motors with gears

When selecting drivers in robotics there are two main options to consider, servomechanism or stepper motor. The servomechanism is every DC or AC motor with closed control loop, for example, brushless motor with encoder and PID controller which allows it to reach the desired position. The second type is a stepper motor. This driver does not require feedback, it can be controlled in an open loop, as it does not rotate freely, but takes steps. This type of motor is more common in hobby designs, as it is easier to use and cheaper.

In this project, both stepper and servomotors are used. The first five axes are equipped with Nema23 and Nema17 stepper motors, while the sixth axis and gripper use servo 180. Nema stepper motors were chosen as they have a high torque while being affordable. Every stepper motor has 200 steps per revolution [6] and is set to work with 1/16 microstepping. In addition, to achieve higher precision, gear ratios were used for all axes. The resulting resolution for each joint is summarized below:

Table 4. Kinematic parameters and step resolution for each axis.

Axis	Motor Type	Gear Ratio	Steps/Rev	Resolution [°/step]
1	Nema17 HS4401	8:1	25,600	0.01406
2	Nema23 57HS112-3004	128:20	20,480	0.01758
3	Nema17 HS4401	8:1	25,600	0.01406
4	Nema17 HS4401	1:1	3,200	0.11250
5	Nema17 HS4023	4:1	12,800	0.02812
6	Servo SG995	1:1	N/A	N/A

Based on the data in Table 4 some observations can be made. The precision on the first three axes, which are responsible for positioning the wrist of the robot in a space, is less than 0.02° per step, which is more than enough for simple pick and place operations. The fourth axis has the lowest precision and no gear ratio, because it only rotates the forearm of the robot. The sixth axis is equipped with a servomotor that can rotate in 180° and its resolution is based mainly on applied control via PWM signal.

Another aspect of gear ratio is the increase in a torque liniary to the ratio applied. The torques for each motor can be seen in the following table:

Table 5. The torques of each motor. [6], [9], [10], [7]

Axis	Motor Type	Motor's torque [Nm]	Gear Ratio	Joint's torque [Nm]
1	Nema17 HS4401	0.4	8:1	3.2
2	Nema23 57HS112-3004	3	128:20	19.2
3	Nema17 HS4401	0.4	8:1	3.2
4	Nema17 HS4401	0.4	1:1	0.4
5	Nema17 HS4023	0.14	4:1	1.6
6	Servo SG995	1	1:1	1

Motors and gear ratios were chosen independently for each joint. As the second axis holds the whole robotic arm it had to be the strongest one. The first, third and fourth axes were given the same motor, but different gear ratios. Although the first and third have some load to overcome, the fourth joint does not need much strength since it only spins the forearm. The fifth axis was equipped with a smaller Nema17 motor to reduce weight. The last sixth axis has serwo 180°. With the assumption that every gripper used in its build is simetrical, there was no need for a more advanced motor.

5.2 Positioning and feedback

To accurately position the robotic arm in 3D space, it is essential to establish a reference coordinate system. This is achieved with a process known as homing. The manipulator is equipped with five end-stop sensors, one for each axis. During homing, the control system sets the speed of each motor to the slow safety value and sets the direction to move toward the endstop. When end-stop is activated, the link stops and waits for other to finish homing. Once all of the joints are finished, the robot sets this position as home to know the relative orientation of each axes. From now on, the controller tracks the position of the stepper motors based on the amount of steps each took. Since stepper motors provide predictable motion, they can operate effectively without external feedback, assuming that the load does not exceed the motor's holding torque, which would result in lost steps. Similarly, the 180° servomotor does not need external feedback, as its closed loop circuitry maintains the shaft position based on the width of the incoming PWM (Pulse Width Modulation) signal.

5.3 Control unit - uC, drivers

The STM32H755ZI-Q microcontroller was selected as the central processing unit for the robotic manipulator. It is an extremely powerful board characterized by the following parameters [11]:

- **Dual-core architecture:** Featuring Cortex-M7 and Cortex-M4 cores with a clock speed of up to 480 MHz. This high computational power, supported by a hardware Floating Point Unit (FPU), is essential for real time Inverse Kinematics (IK) calculations and trajectory planning.
- **Extensive I/O capabilities:** Up to 114 I/O ports with interrupt support. This allows for future upgrades, such as implementing closed loop control with encoders on each axis, adding extra sensors, or integrating manual control interfaces.
- **Advanced Timers (PWM):** High resolution timers allow for precise PWM generation to control servos and provide stable interrupt signals for stepper motor drivers, to keep the motor speeds as it was planned.
- **Communication Protocols:** A wide range of interfaces including I2C, SPI, and UART, which was used in this project for remote robot control.

While simpler controllers would be suitable for this application, this high-performance board is ideal for future expanding the robot's capabilities.

Each stepper motor is equipped with its own driver. For the Nema23 motor on the second axis, a DM556 driver was used to handle a higher current and provide greater torque. The remaining axes use TMC2209 drivers, chosen for their quiet operation and smooth motion control.

5.4 Electronic diagram

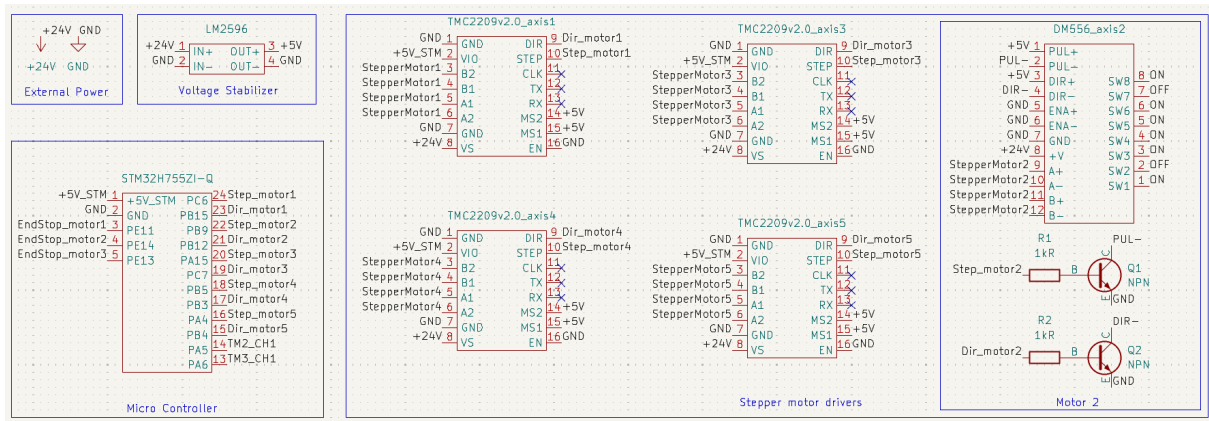


Figure 8. Electrical scheme. Source: Own work

6 Mathematical model

This chapter is based on the work presented in [12].

6.1 Forward kinematics

To successfully control a robotic manipulator, it is essential to understand how the arrangement of each joint affects the position and orientation of the end effector. Based on the length of each robot's link and the angles at which the joints are set, the exact pose of the manipulator's end effector can be calculated. For 6-DOF robotic arms, the Denavit–Hartenberg convention is used. This method describes rotation and translation between consecutive joints with standardised parameters, as shown in the table below.

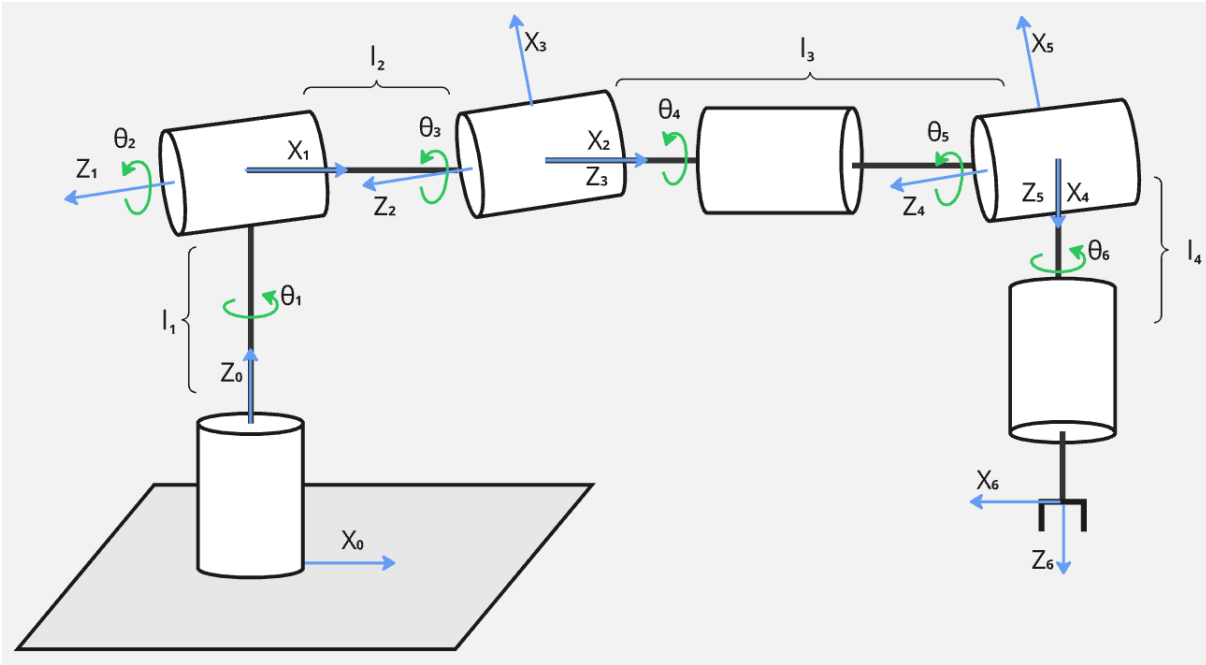


Figure 9. Rotation frames for my robot. Source: Own work

These parameters can be applied to the general D-H matrix (2), which describes the rotation and translation of the joint-link pair.

$$T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

By substituting the values into the matrix, we get the following 6 matrices.

Table 6. Denavit–Hartenberg parameters for 6-DOF robotic arm.

i	a_i	α_i	d_i	θ_i
1	0	$+\frac{\pi}{2}$	l_1	θ_1
2	l_2	0	0	θ_2
3	l_3	0	0	θ_3
4	0	$+\frac{\pi}{2}$	0	θ_4
5	0	$-\frac{\pi}{2}$	0	θ_5
6	0	0	l_4	θ_6

$$T_1 = \begin{bmatrix} \cos \theta_1 & 0 & \sin \theta_1 & 0 \\ \sin \theta_1 & 0 & -\cos \theta_1 & 0 \\ 0 & 1 & 0 & l_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$T_2 = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 & l_2 \cos \theta_2 \\ \sin \theta_2 & \cos \theta_2 & 0 & l_2 \sin \theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

$$T_3 = \begin{bmatrix} \cos \theta_3 & -\sin \theta_3 & 0 & l_3 \cos \theta_3 \\ \sin \theta_3 & \cos \theta_3 & 0 & l_3 \sin \theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

$$T_4 = \begin{bmatrix} \cos \theta_4 & 0 & \sin \theta_4 & 0 \\ \sin \theta_4 & 0 & -\cos \theta_4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

$$T_5 = \begin{bmatrix} \cos \theta_5 & 0 & -\sin \theta_5 & 0 \\ \sin \theta_5 & 0 & \cos \theta_5 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

$$T_6 = \begin{bmatrix} \cos \theta_6 & -\sin \theta_6 & 0 & 0 \\ \sin \theta_6 & \cos \theta_6 & 0 & 0 \\ 0 & 0 & 1 & l_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

Because the movement of the earlier joints influences the pose of all of the following joints, multiplying the matrices results in the final homogeneous transformation matrix defining the end effector pose relative to the base of the robot.

$$T_6^0 = T_1 T_2 T_3 T_4 T_5 T_6 = \begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{p}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}, \quad (9)$$

Where $R_{3 \times 3}$ is a rotation matrix and $p_{3 \times 1}$ is a vector of the position of the end effector.

6.2 Inverse kinematics

When dealing with robotic manipulators, one of the fundamental problems is determining the angles of each joint to achieve the required position and orientation of the end effector. This problem, known as inverse kinematics, is not trivial. Firstly, there can be more than one solution or no solution at all for the given configuration of the end effector. Additionally, not every manipulator has an analytical closed form solution for the inverse kinematics. There are two conditions that must be met. The first one is that three of the manipulator's joints must be parallel. The second is that there needs to be a spherical joint, which means that three of its joints must have a rotation axis that intersects in a common point. The robot described in this thesis meets those requirements, which allows the inverse kinematics problem to be solved.

As a starting point, the homogeneous transformation matrix T_6^0 is given. As explained in the previous chapter, this matrix contains a rotation matrix R_6^0 and a position vector P_6^0 of the end effector.

Due to the presence of a spherical wrist, the inverse kinematics problem can be divided into two independent ones. The first three joints are responsible for the position of the robot's wrist, while the last three define its orientation. With that, the problem comes to computing the position of the wrist and the orientation R_3^6 of the last three joints.

First, by moving one joint back from the known location of the end effector, the exact position of the wrist can be calculated.

$$\mathbf{W} = \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix} = \begin{bmatrix} p_x - d_6 r_{13} \\ p_y - d_6 r_{23} \\ p_z - d_6 r_{33} \end{bmatrix} \quad (10)$$

Here, the position is taken from $P_6^0 = [p_x, p_y, p_z]^T$ and r_{nm} is taken from R_6^0 .

6.2.1 Position of the wrist

With the wrist pose known, the first part of the problem can simply be referred to as follows.

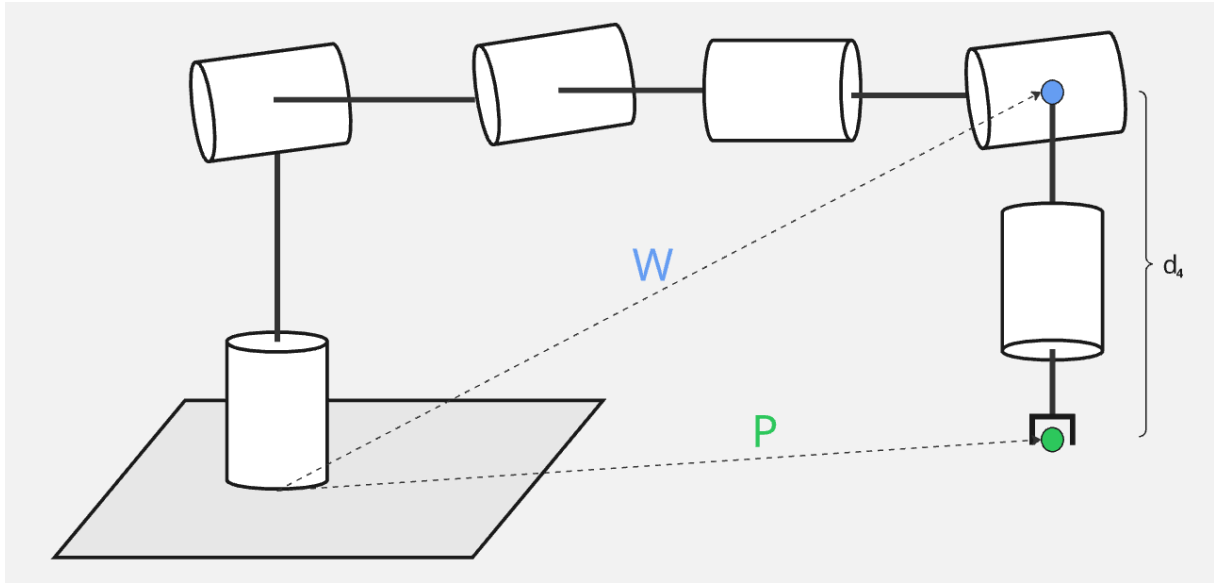


Figure 10. Correlation between end effector position and the wrist. Source: Own work

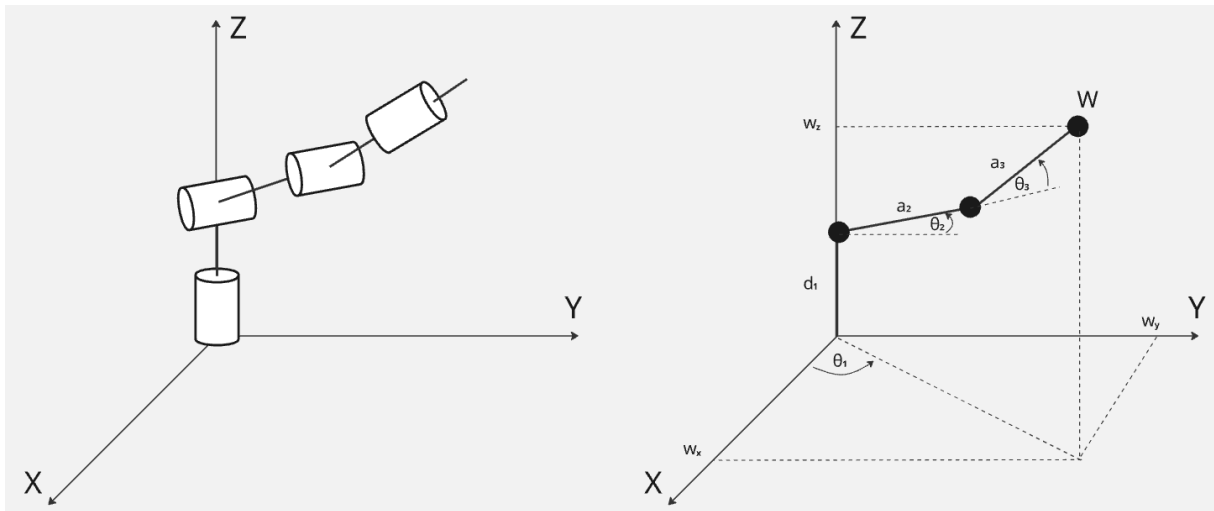


Figure 11. First three axis of the robotic arm. Source: Own work

Based on fig. 11, θ_1 can be expressed as

$$\theta_1 = \tan^{-1} \left(\frac{w_y}{w_x} \right) \quad (11)$$

To determine θ_2 and θ_3 , the 2D plane should be prepared as presented in fig. 12.

Where:

$$r = \sqrt{w_x^2 + w_y^2}, \quad s = w_z - d_1 \quad (12)$$

Then, the law of cosines (13) can be applied to find θ_3 .

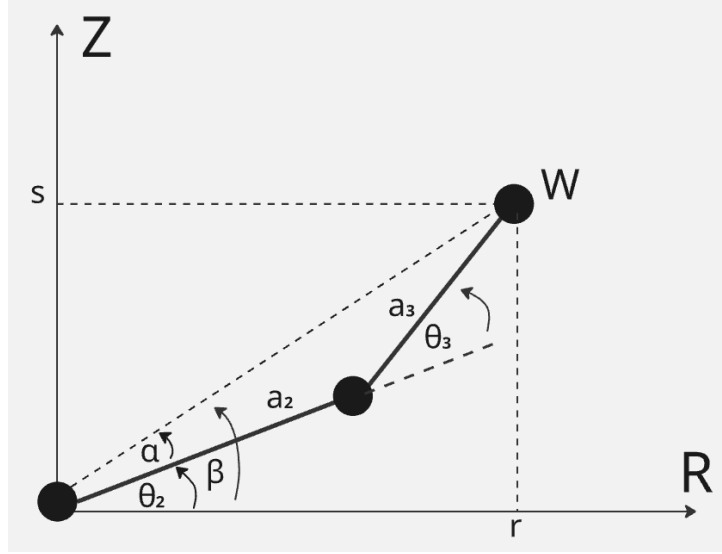


Figure 12. 2D plane with first and second manipulator's arms. Source: Own work

$$c^2 = a^2 + b^2 - 2ab \cos \gamma \quad (13)$$

With values assigned earlier, the final equation takes the following form:

$$\theta_3 = \cos^{-1} \left(\frac{r^2 + s^2 - a_2^2 - a_3^2}{2a_2a_3} \right) \quad (14)$$

The final step is to determine θ_2 , which can be calculated as $\theta_2 = \alpha - \beta$, where the angles are defined as

$$\alpha = \text{atan2}(s, r) \quad (15)$$

$$\beta = \text{atan2}(a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3) \quad (16)$$

$$\theta_2 = \text{atan2}(s, r) - \text{atan2}(a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3) \quad (17)$$

6.2.2 Orientation of the wrist

Now, the wrist orientation must be calculated. It is represented by the matrix R_6^3 , which can be calculated from:

$$R_6^3 = (R_3^0)^T R_6^0 \quad (18)$$

The R_6^0 is known because it is our desired orientation. The R_3^0 can be taken from T_3^0 , which is the multiplication of the matrices $T_{1..3}$ from the previous chapter and is equal to

$$T_0^3 = \begin{bmatrix} c_1 c_{23} & -c_1 s_{23} & s_1 & w_x \\ s_1 c_{23} & -s_1 s_{23} & -c_1 & w_y \\ s_{23} & c_{23} & 0 & w_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (19)$$

The same can be done with the matrix R_6^3 , as it is a part of the transformation matrix T_6^3 , which is a multiplication of matrices $T_{4..6}$.

$$T_6^3 = \begin{bmatrix} c_4 c_5 c_6 - s_4 s_6 & -c_4 c_5 s_6 - s_4 c_6 & c_4 s_5 & p_x \\ s_4 c_5 c_6 + c_4 s_6 & -s_4 c_5 s_6 + c_4 c_6 & s_4 s_5 & p_y \\ -s_5 c_6 & s_5 s_6 & c_5 & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (20)$$

Now, substituting the rotation matrices into the equation (18) and performing the transformation, the following equation is obtained.

$$\begin{bmatrix} c_4 c_5 c_6 - s_4 s_6 & -c_4 c_5 s_6 - s_4 c_6 & c_4 s_5 \\ s_4 c_5 c_6 + c_4 s_6 & -s_4 c_5 s_6 + c_4 c_6 & s_4 s_5 \\ -s_5 c_6 & s_5 s_6 & c_5 \end{bmatrix} = \begin{bmatrix} c_1 c_{23} & s_1 c_{23} & s_{23} \\ -c_1 s_{23} & -s_1 s_{23} & c_{23} \\ s_1 & -c_1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \quad (21)$$

From this equality five crucial equations can be written:

$$c_4 s_5 = c_1 c_{23} r_{13} + s_1 c_{23} r_{23} + s_{23} r_{33} \quad (22)$$

$$s_4 s_5 = -c_1 s_{23} r_{13} - s_1 s_{23} r_{23} + c_{23} r_{33} \quad (23)$$

$$c_5 = s_1 r_{13} - c_1 s_{23} \quad (24)$$

$$s_5 s_6 = s_1 r_{12} - c_1 r_{22} \quad (25)$$

$$-s_5 c_6 = s_1 r_{11} - c_1 r_{22} \quad (26)$$

Now, the three remaining angles θ_4 , θ_5 , and θ_6 depend only on the angles determined previously. For θ_5 first the $\sin(\theta_5)$ can be calculated and then used in the atan2 function.

$$s_5 = \sqrt{1 - c_5^2} \quad (27)$$

$$\theta_5 = \text{atan2}\left(\pm \sqrt{1 - (s_1 r_{13} - c_1 r_{23})^2}, s_1 r_{13} - c_1 r_{23}\right) \quad (28)$$

By dividing equations (23) and (22), θ_4 can be obtained.

$$\theta_4 = \text{atan2}\left(-c_1 s_{23} r_{13} - s_1 s_{23} r_{23} + c_{23} r_{33}, c_1 c_{23} r_{13} + s_1 c_{23} r_{23} + s_{23} r_{33}\right) \quad (29)$$

Identical for the last angle, by taking (25) and (26) and dividing them, θ_6 can be obtained:

$$\theta_6 = \text{atan2}(s_1 r_{12} - c_1 r_{22}, -s_1 r_{11} + c_1 r_{22}) \quad (30)$$

6.3 Summary of inverse kinematics and constraints

The equations presented in the following allow to calculate the required angles for all six joints.

$$\theta_1 = \tan^{-1}\left(\frac{w_y}{w_x}\right) \quad (11)$$

$$\theta_2 = \text{atan2}(s, r) - \text{atan2}(a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3) \quad (17)$$

$$\theta_3 = \cos^{-1}\left(\frac{r^2 + s^2 - a_2^2 - a_3^2}{2a_2 a_3}\right) \quad (14)$$

$$\theta_4 = \text{atan2}\left(-c_1 s_{23} r_{13} - s_1 s_{23} r_{23} + c_{23} r_{33}, c_1 c_{23} r_{13} + s_1 c_{23} r_{23} + s_{23} r_{33}\right) \quad (29)$$

$$\theta_5 = \text{atan2}\left(\pm \sqrt{1 - (s_1 r_{13} - c_1 r_{23})^2}, s_1 r_{13} - c_1 r_{23}\right) \quad (28)$$

$$\theta_6 = \text{atan2}(s_1 r_{12} - c_1 r_{22}, -s_1 r_{11} + c_1 r_{22}) \quad (30)$$

In theory, this system of equations can result in up to 8 different sets of solutions for a single target position. For example, the robot could reach a point with its "elbow" pointing up or down. To make sure that the robot's motion is predictable, there is an algorithm that always prioritizes the same solution from the available ones. This means that if both solutions are allowed by the set limits of the robot, the software will always choose the same one.

In this project, no joint rotates by more than 180 degrees. This restriction is mainly made to ensure that cables do not get tangled, but it also helps the IK by removing some of the theoretical solutions. The operational ranges for each joint are presented in the table below.

Table 7. Software and mechanical limits for each joint.

Joint	Minimum Angle	Maximum Angle
θ_1 (Base)	-90.0°	45.0°
θ_2 (Shoulder)	10.0°	135.0°
θ_3 (Elbow)	-170.0°	0.0°
θ_4 (Wrist Roll)	-90.0°	90.0°
θ_5 (Wrist Pitch)	-90.0°	90.0°
θ_6 (Wrist Yaw)	-90.0°	90.0°

Another critical aspect of the movement is the presence of **singularities**. A singularity occurs when two or more axes align in a straight line, causing the robot to lose a degree of freedom. The most common example is the position where the wrist is located directly above the base. In this state, the calculation for θ_1 becomes unstable because any rotation of the base results in the same tool position. To ensure correct operation, the software constantly checks for known singularities and prevents the robot from entering them.

6.4 Algorithms of TCP control

The Tool Center Point (TCP) algorithm is a control method designed to move a robotic arm's end effector along a specific trajectory while keeping the pose or orientation. This is essential in practical applications, such as

- **Palletization tasks:** Where a robot must move an object from point A to point B while keeping it perfectly level or at a specific angle.
- **Welding or gluing:** Where the distance and angle to the surface must be kept constant throughout the whole movement.

To make the algorithm work correctly, there are three functions that need to be implemented:

6.4.1 Path linearization and interpolation

Since the main goal of TCP movement is controlling the trajectory, the desired path needs to be "forced" onto the robotic arm. This is done by dividing the long path into many small segments. Without this division, the manipulator would move in a trajectory that is efficient for its joints but completely unpredictable for the user. Using smaller steps, the robot is forced to visit each point in sequence. If these points are close enough together, the robot basically moves in the exact straight line it was told to follow. Additionally, managing these shorter steps makes it much easier to keep the end effector in the desired orientation throughout the whole move.

6.4.2 Joint synchronization

Once the path is divided into microsteps, the movements of all six joints must be synchronized. If the joints operated independently, some might reach their target position faster than others, resulting in a jittery motion where the robot repeatedly stops and starts. In a synchronized system, the controller calculates the necessary velocity for each axis so that they all start and finish each microstep simultaneously. This ensures a smooth, continuous motion of the Tool Center Point.

In this project, the synchronization was implemented by choosing the leading joint. The process follows these steps:

- **Step to angle conversion:** First, the software takes the target angles from the IK equations and calculates the exact number of steps each motor must perform.
- **Identifying the bottleneck:** Using the predefined maximum safe speed for each motor, the controller calculates the minimum time required for each joint to complete its portion of the move:

$$t_{joint} = \frac{\text{steps to perform}}{\text{max speed [steps/s]}}$$

- **Leading joint selection:** The joint that requires the most time to finish its move becomes the leading joint. This joint will move at its maximum allowed speed, acting as the speed bottleneck for the entire system.
- **Velocity scaling:** To ensure all motors finish at the exact same time, the speeds of the remaining five joints are scaled down proportionally:

$$\text{Target Speed}_n = \frac{\text{steps}_n}{t_{lead}}$$

This approach ensures two things. First, all movements are safe because no motor ever exceeds its set speed limit. Second, all joints start and finish their rotation at the same time, eliminating the need for any motor to stop and wait, resulting in smooth continuous motion.

6.4.3 Look-ahead and velocity planning

To achieve smooth motion, the robot should avoid unnecessary acceleration and deceleration at every intermediate point. This is managed through Look-Ahead planning. Instead of just looking at the current move, the algorithm calculates several points in advance to analyze the next segments of the path. It checks each individual joint to see if it needs to continue moving in the same direction or not. If the direction stays the same, the controller prevents the motor from decelerating and maintains its current speed. However, if an axis needs to stop or change direction in the next step, the system starts to decelerate early to prepare for that change. This approach allows the robot to operate at its maximum efficient speed while maintaining mechanical stability and avoiding jerky movements.

7 Software

Several different software tools were used to complete this thesis. Programs like FreeCAD, Bambulab Studio, and KiCad were used mainly for mechanical and electrical design. On the other hand, Python and STM32CubeIDE were chosen to handle the robot's control system and all its functionalities.

7.1 Programming languages used

Two main programming languages were used in this project: Python for the high-level control on the PC, and C for the low-level logic running on the STM32 mainboard.

7.1.1 Python

Python was chosen because it is one of the best languages for creating a Graphical User Interface (GUI) for easy remote control. It offers numerous libraries that simplify development. One of them is *Tkinter*, which was used to set up the control window with buttons, sliders, and text boxes. Additionally, the *NumPy* library was used to handle all the complex matrix calculations needed for the robot's movement. To make the system easier to debug, the *Matplotlib* library was also implemented to create a 3D simulation of the robot. This allows the user to visualize and test the trajectory on the screen before the real robotic arm starts moving.

7.1.2 C

C was chosen as it is the main language used in STM32CubeIDE. It is a very efficient and fast language that gives the programmer great control over the hardware. This speed is essential for a robot controller that needs to work in real-time, especially when calculating Inverse Kinematics (IK) and checking safety constraints simultaneously. Using C also allows for direct management of the microcontroller's internal timers and interrupts, which is crucial for generating precise steps for the motors and responding to sensor data without any delay.

7.2 Communication computer - robot

Several communication methods were possible, but a serial connection (UART) was chosen for this project. It is easy to set up over a standard USB port that connects the computer directly to the STM32 board. This standard is fast enough to ensure that there is no visible delay between sending a command and the robot performing the movement. A baud rate of 115200 was selected as it offers a great balance between communication speed and data reliability.

Most of the communication is one-way, following a "command-action" structure. The computer sends a specific data packet that the board receives, processes, and then executes. To ensure that the robot knows exactly when a full command has arrived, each string is terminated with a newline character ($\backslash n$). This acts as a trigger for the STM32 to start processing the received data.

However, for certain tasks, two-way communication is necessary to ensure that the system stays synchronized:

- **Homing procedure:** The computer sends the homing command and waits for a "ready" confirmation from the board once all axes have reached their limit switches.
- **IK movement:** After receiving the coordinates, the STM32 notifies the computer when the movement is finished. This prevents the PC from sending new coordinates before the robot is ready for the next step.

7.3 Graphical User Interface (GUI)

To make the robot easy to operate, a dedicated Graphical User Interface (GUI) was developed using Python and the Tkinter library. The interface allows the user to monitor the robot's status and choose between different control modes. At the very top, a status indicator shows whether the robot is "Ready" or "Busy" (occupied).

As shown in Figure 13, the interface is divided into several sections:

1. **Inverse Kinematics (IK) Control:** This section allows the user to input Cartesian coordinates (X, Y, Z) and orientation angles (Roll, Pitch, Yaw). Once the "Perform IK Move" button is pressed, the PC sends the raw data to the STM32 which calculates how to move the arm to that exact point.
2. **Demo Mode:** This part is used to trigger pre-programmed demonstration sequences. It is particularly useful for showing smooth TCP movements along a chosen axis without manual input.
3. **Homing:** The large orange button starts the homing procedure. This must be used every time the robot is powered on so that the controller can find the "zero" position for each joint using the limit switches.
4. **Manual Joint Control:** This section allows for the movement of each of the first five axes individually. The user can also adjust the "delay" value, which directly changes the speed of the stepper motors by increasing or decreasing the time between steps.

5. **Accessories (Servos):** These sliders control the two servos located at the end of the arm. One slider rotates the sixth axis (the wrist), and the other opens or closes the gripper.
6. **Vision System and Calibration:** This section handles camera based control. It allows the user to start the camera, calibrate the robot's coordinate system with the camera's view (using ArUco markers), and enable hand tracking mode.
7. **Relative Step Debugging:** This is a low level tool used for testing and maintenance. It allows the user to move a specific motor by a precise number of steps in either direction.

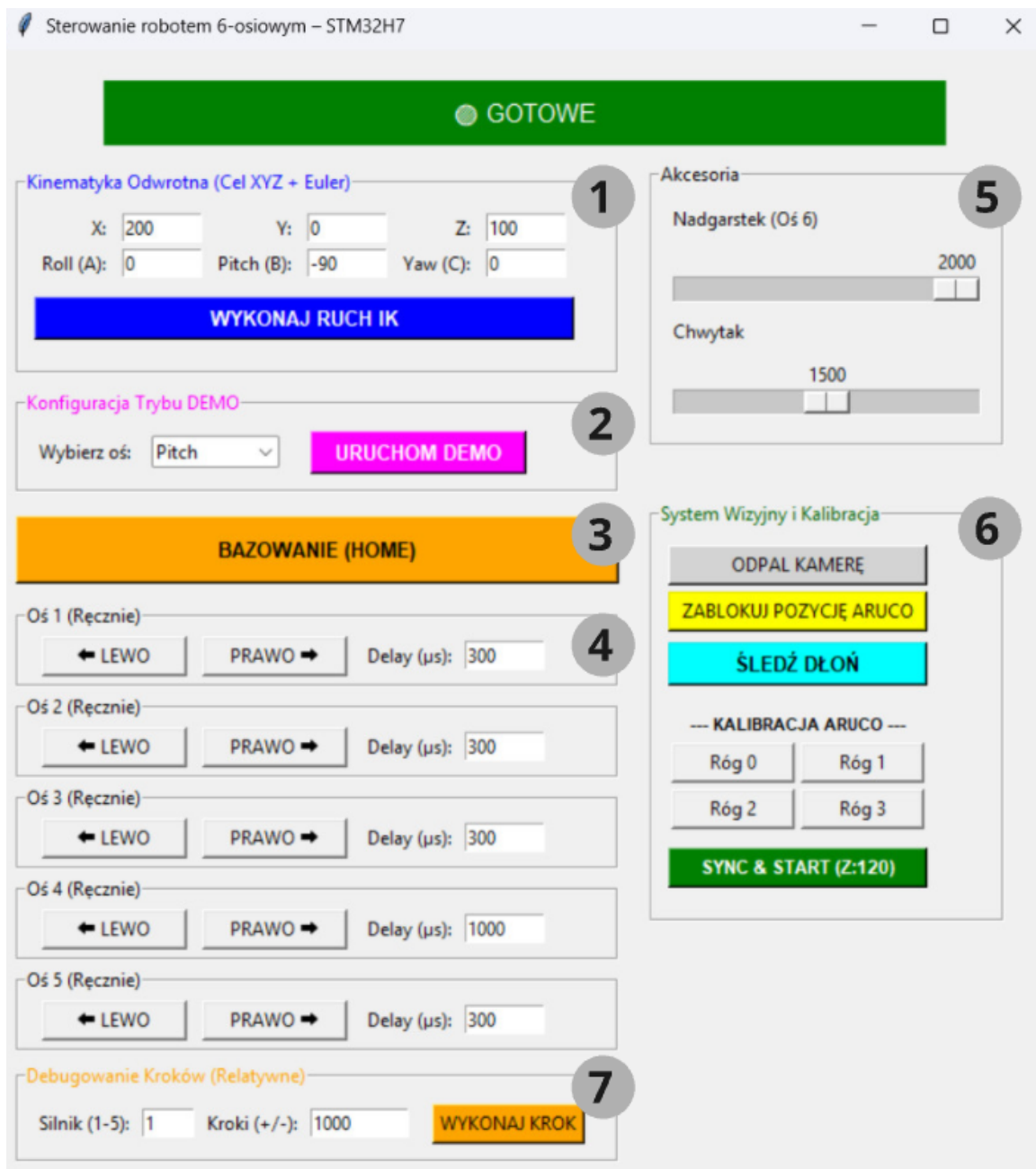


Figure 13. The Graphical User Interface designed for controlling the robotic arm. Source: Own work

7.4 Simulation and IK debugging

Before testing inverse kinematics on the physical robot, a simulation tool was created using the *Matplotlib* library. This tool was crucial for debugging, as it allowed a safe verification of the mathematical model. The simulation displays the robot's pose from four different perspectives: a 3D view and three 2D projections (top, side, and front).

The user can input the coordinates (X, Y, Z) and orientation angles (Roll, Pitch, Yaw) to see how the virtual arm positions itself. At the bottom of the interface, the program displays the number of motor steps that each axis needs, to reach that position from the home state.

This function was extremely helpful during the testing. This allowed for a comparison. The coordinates were entered into the simulation and then each joint of the real robot was moved by the calculated number of steps. By checking if the real end effector ended up in the same spot as the virtual one, the accuracy of the IK algorithm could be confirmed with the reduced risk of accidental collisions.

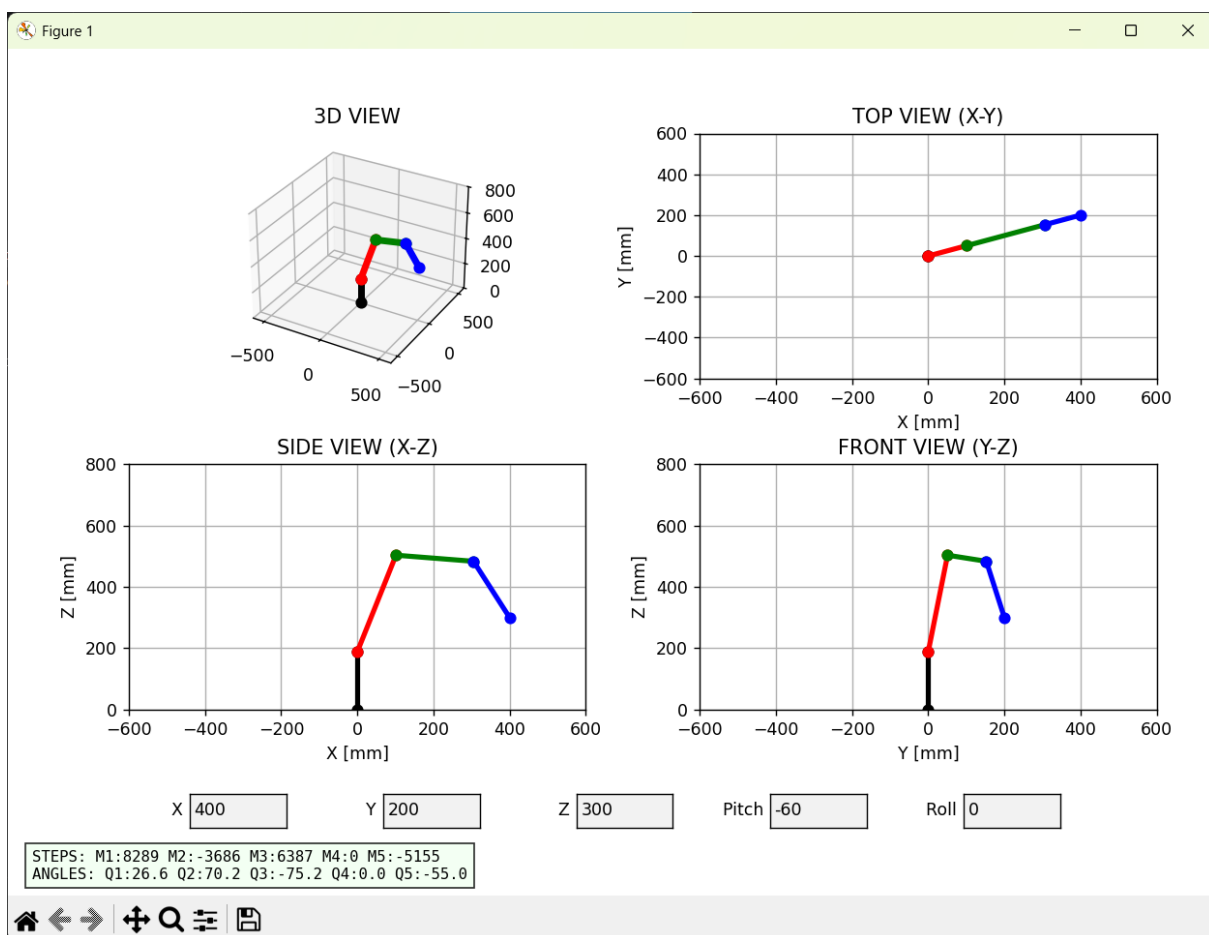


Figure 14. Python simulation used for Inverse Kinematics verification and debugging. Source: Own work

8 Validation and testing

8.1 Load tests

To evaluate the physical capabilities of the robot, a series of load tests were conducted. This experiment aimed to determine the maximum payload that the arm could carry at its maximum reach. The experimental setup is shown in Figure 15.

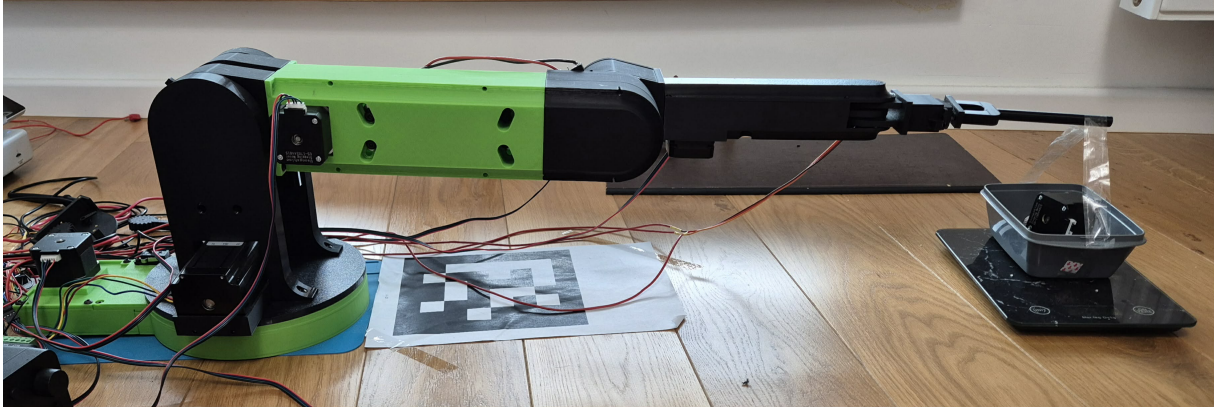


Figure 15. Experimental setup for measuring the robot's lifting capacity. Source: Own work

The main bottlenecks in the robot's lifting capacity are the second joint (shoulder) and the third joint (elbow), as they must overcome the largest torque. For this reason, the test was split into two independent tests. Each test focused on one of these joints while keeping all other axes in a locked constant position.

The tests analyzed the impact of two key variables: the microstepping ratio and the peak current supplied to the stepper motor driver.

8.1.1 Shoulder joint (joint 2) evaluation

In the second joint, a NEMA 23 stepper motor is used and controlled by a DM556 driver. During the test, the microstep ratio was compared between a full step (1:1) and a microstep (1:16), which corresponds to 200 and 3200 steps per revolution, respectively. In addition, two peak current limits were tested: a lower value of 2.7 A and a higher value of 4.9 A. The results of these measurements are presented in Table 8.

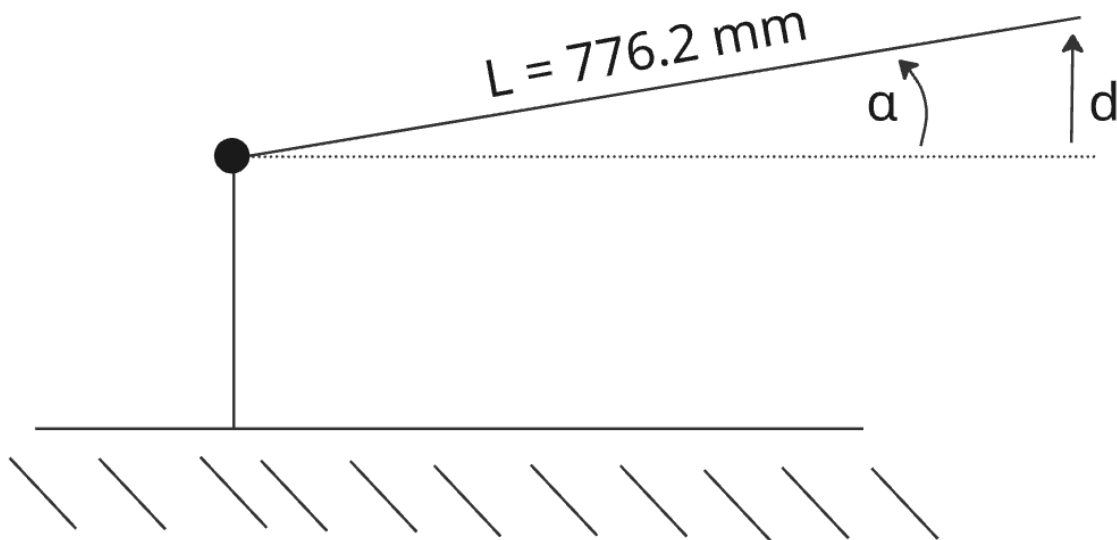
Table 8. Load capacity test results for Joint 2 (Shoulder).

Joint	Microstepping	Peak Current [A]	Max Payload [g]
2	1:1	2.7	130
2	1:1	4.9	150
2	1:16	2.7	150
2	1:16	4.9	180

The payload values in the table represent the rounded average of several measurements. As expected, supplying a higher current to the motor increased its torque, allowing the arm to lift more weight. However, doubling the peak current from 2.7 A to 4.9 A resulted in only about a 15% to 20% increase in lifting capacity.

An interesting finding was the impact of microstepping. Changing the driver setting from full steps (1:1) to (1:16) microsteps increased the lifting strength by approximately 15% at 2.7 A and 20% at 4.9 A. Theoretically, increasing the microstepping ratio should decrease the holding torque because the rotor is forced to hold positions between the magnetic poles. However, in this setup, microstepping significantly reduced vibrations and jerk.

To show why this happens, the distance that the end effector moves during a single step can be calculated.

**Figure 16.** Displacement of end effector. Source: Own work

For a full step (1:1) mode, the motor rotates by $\alpha_{1:1} = 360^\circ/200 = 1.8^\circ$ per step. Assuming a straight-line approximation for such a small angle, the displacement $d_{1:1}$ of the robot's end

effector during a single step is:

$$d_{1:1} = L \cdot \sin(\alpha_{1:1}) = 776.2 \text{ mm} \cdot \sin(1.8^\circ) \approx 24.38 \text{ mm} \quad (31)$$

For the (1:16) microstepping mode, the step angle is reduced to $\alpha_{1:16} = 360^\circ/3200 = 0.1125^\circ$ per step. This results in a displacement $d_{1:16}$ of:

$$d_{1:16} = L \cdot \sin(\alpha_{1:16}) = 776.2 \text{ mm} \cdot \sin(0.1125^\circ) \approx 1.52 \text{ mm} \quad (32)$$

This shows that, in full-step mode, each single step tries to move the end effector by almost 25 mm. Because the arm is 3D-printed, this sudden displacement generates huge forces, making the arm to wobble and causing the motor to fail under lighter loads than it should. With the (1:16) microstepping ratio, the distance is just 1.5 mm, which makes the movement smoother and reduces the shaking. This allows the motor to use all its torque more efficiently.

8.1.2 Elbow joint (joint 3) evaluation

In the third joint, a NEMA 17 stepper motor is used and controlled by a TMC2209 driver. During the test, the microstep ratio was compared between a microstep ratio of (1:8) and (1:64), which corresponds to 1600 and 12800 steps per revolution, respectively. In addition, the reference voltage was tested from a power efficient value to the maximum, safe for the motor value [6]. The reference voltage can be calculated [13] with the equation shown below.

$$V_{REF} = I_{RSM} \cdot \sqrt{2} \quad (33)$$

The results of these measurements are presented in Table 9.

Table 9. Load capacity test results for Joint 3 (Elbow).

Joint	Microstepping	V_{REF} [V]	I_{RSM} [A]	Max Payload [g]
3	1:8	1.0	0.71	140
3	1:8	1.4	0.99	220
3	1:8	1.8	1.27	300
3	1:8	2.2	1.56	420
3	1:64	1.0	0.71	150
3	1:64	1.4	0.99	250
3	1:64	1.8	1.27	340
3	1:64	2.2	1.56	420

The measurements show an almost linear relationship between the driver's reference voltage and the strength of the robot, which was expected.

Similarly to the shoulder joint, increasing microstepping resulted in a slightly stronger manipulator. Moving from (1:8) to (1:64) microstepping mode improved the lifting capacity by about 7% at 1.0 V, 13% at 1.4 V, and 13% at 1.8 V.

This improvement can again be explained by analyzing the displacement at the end effector. The distance from the axis of Joint 3 (elbow) to the end of the gripper is $L_{elbow} = 440.6$ mm.

For a (1:8) microstep, the joint step angle is $\alpha_{1:8} = 360^\circ/1600 = 0.225^\circ$. The displacement $d_{1:8}$ of the end effector during a single step is:

$$d_{1:8} = L_{elbow} \cdot \sin(\alpha_{1:8}) = 440.6 \text{ mm} \cdot \sin(0.225^\circ) \approx 1.73 \text{ mm} \quad (34)$$

For a (1:64) microstep, the joint step angle is $\alpha_{1:64} = 360^\circ/12800 = 0.0281^\circ$, which gives the displacement $d_{1:64}$ of:

$$d_{1:64} = L_{elbow} \cdot \sin(\alpha_{1:64}) = 440.6 \text{ mm} \cdot \sin(0.0281^\circ) \approx 0.22 \text{ mm} \quad (35)$$

This again shows that when moving a single step, the higher value of microstepping reduces the displacement of the end effector more. That ensures smoother operation, which reduces jerk and increases strength.

8.2 TCP trajectory test

To evaluate path planning and joint synchronization, two Tool Center Point (TCP) trajectories, linear and semicircular, were tested and compared between the Python simulation and the physical robot.

Both movements have the same starting and ending poses. The coordinates and orientation angles for both tests are defined as follows.

- **Start Pose:** $X = 400$ mm, $Y = 0$ mm, $Z = 0$ mm, Roll = 0° , Pitch = -60° , Yaw = 0° .
- **End Pose:** $X = 600$ mm, $Y = 0$ mm, $Z = 0$ mm, Roll = 0° , Pitch = -60° , Yaw = 0° .

8.2.1 Linear trajectory

In the first test, the end effector was moved in a straight line between the start and the end point. The orientation was kept constant throughout the whole path. For the simulation, only several points were calculated to illustrate the intermediate poses, but for the movement of the physical robot, the entire path was split into small steps to keep the desired path throughout the entire distance.

The following is a comparison of the trajectory generated in the simulation and the actual physical movement of the robotic arm.

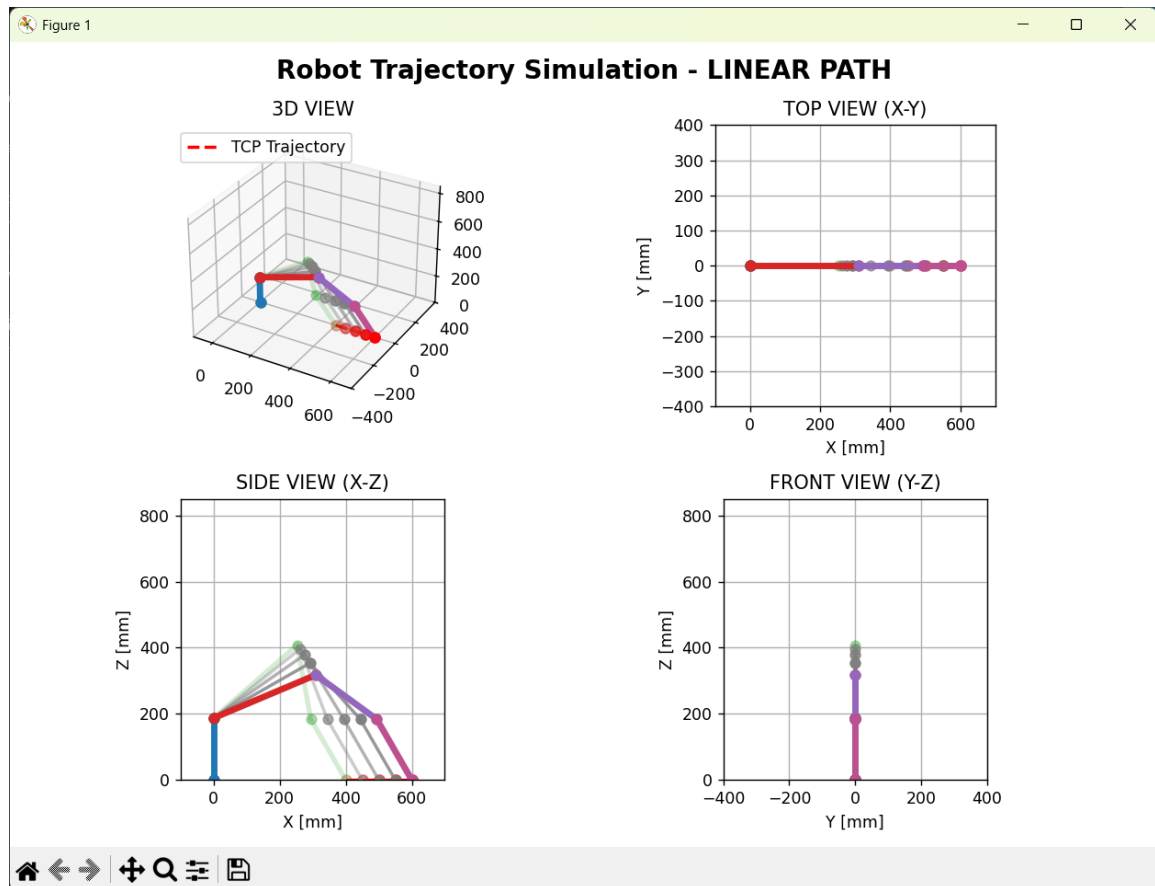


Figure 17. Python simulation showing the poses of the robotic arm during the linear trajectory. Source: Own work

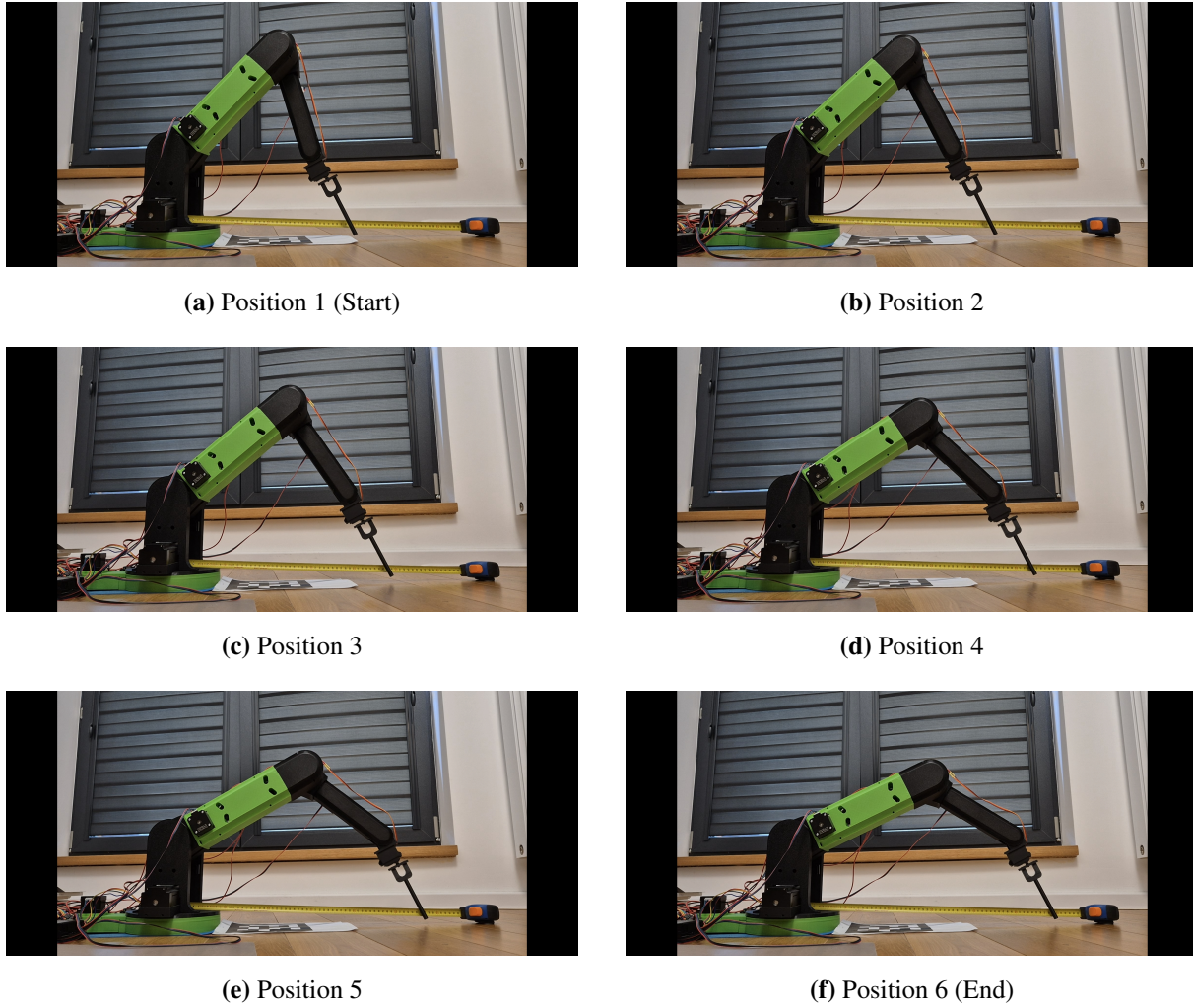


Figure 18. Steps of the physical robotic arm executing the linear trajectory. Source: Own work

8.2.2 Semicircular trajectory

In the second test, the end effector was moved in a semicircular arc between the same coordinates. Again, the orientation was kept constant. Simulation shows several possible configurations in the trajectory while the physical robot traveled the entire path. This test was crucial in verifying how well the system coordinates multiple axes.

The following is a comparison of the trajectory generated in the simulation and the actual physical movement of the robotic arm.

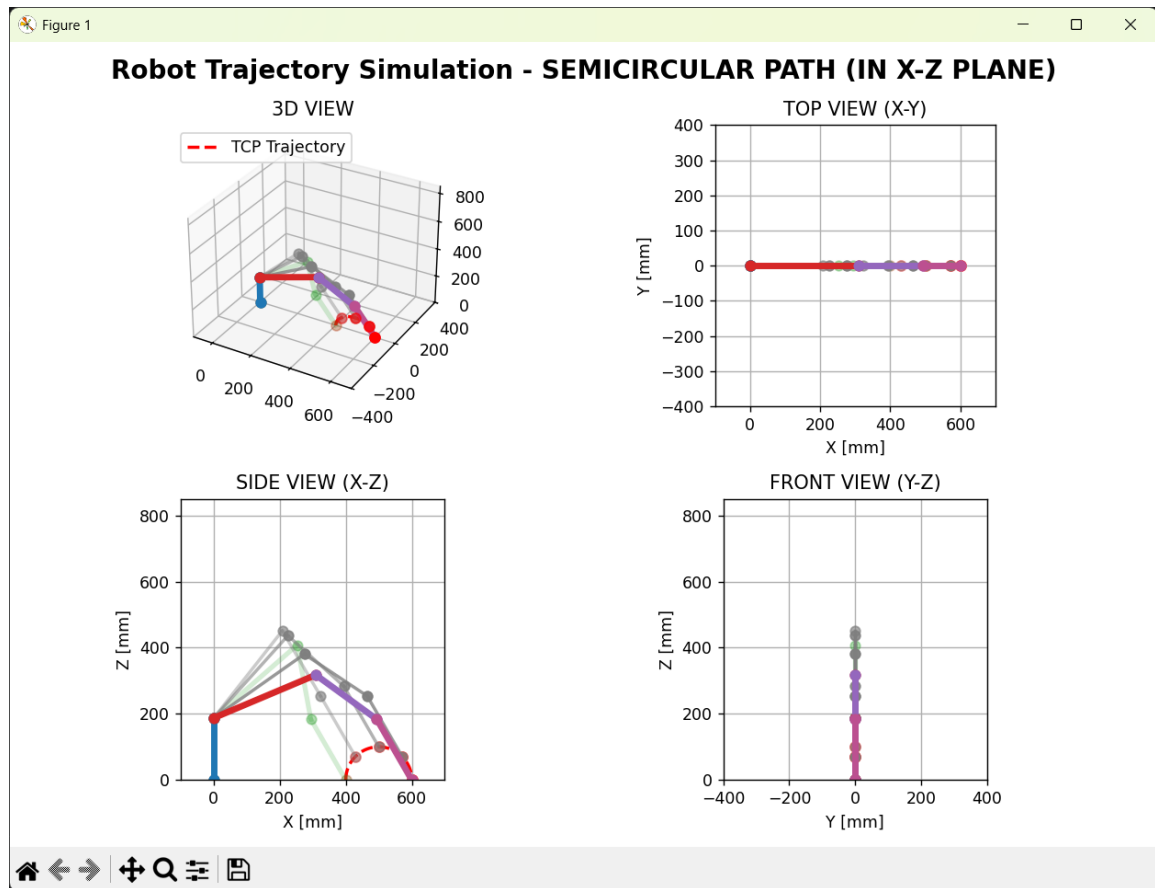


Figure 19. Python simulation showing the sequential poses of the robotic arm during the semicircular trajectory. Source: Own work

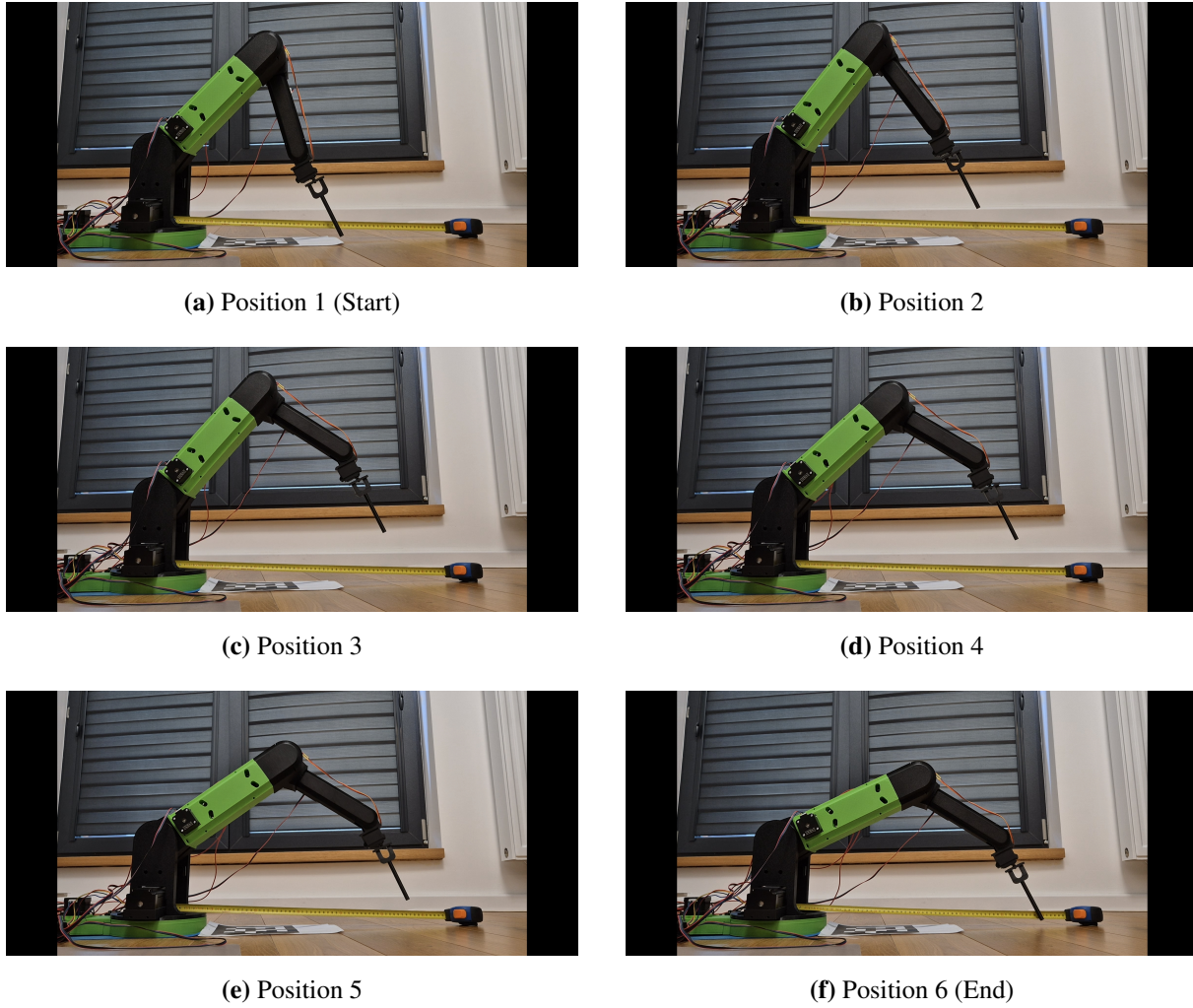


Figure 20. Steps of the physical robotic arm executing the semicircular trajectory. Source: Own work

8.2.3 Trajectory Exceptions

As mentioned in Section 6.3, a single pose can be achieved using 8 different sets of solutions. A prime example of this is the "Elbow Up" and "Elbow Down" configuration. In that case, the second (arm) and third (forearm) joints are mirrored downward. Consequently, the coordinates and orientation remain exactly the same in both cases, but the joint configurations are different. To avoid this ambiguity, a software function prioritizing a single solution was implemented, taking into account the constraints of the robot's physical movements.

9 Conclusions

This thesis showed that it is entirely feasible to design and construct a 6cDof robotic arm using mainly 3D printed parts. The manipulator uses the same kinematics and similar control logic as those found in industrial robots. Due to its lightweight plastic construction, the arm is safe for human interaction, making it a great tool for cooperation, learning, and further experimentation.

9.1 Summary of results

The performance of the robotic arm was evaluated with two experiments, load testing and trajectory verification.

The load tests gave interesting results. Although theoretical models say that full step control gives a holding torque higher than that of microstepping, practical tests proved otherwise. Because full steps produce greater displacement of the end effector, they generate vibrations. In contrast, microstepping provided smoother motion, which prevents the motor from stalling and allowing it to lift higher payloads.

Another interesting finding is the difference between the theoretical maximum strength and the experimental limits. When comparing the torque requirements calculated in Chapter 4.3 with the theoretical torque presented in table 5, one could see that the system should have a large reserve of strength. However, tests showed that a real world model needs to take into account mechanical wear and component flexibility. Since 3D-printed parts are not 100% rigid, a minor flex appears under load, creating additional moments, which decreases the final manipulators strength.

The second test focused on trajectory tracking, where the manipulator performed exceptionally well. The software managed to handle real time inverse kinematics and velocity scaling, allowing the arm to calculate and execute commands without any visible latency. The paths executed by the robot match the trajectories generated in the Python simulation with a margin of backlash error.

9.2 Evaluation of potential usage in real life applicaiton

Based on the experimental results, the robotic arm is perfect for educational and research purposes for schools and universities. It could serve as a tool for teaching future engineers everything regarding robotics, including:

- CAD modeling for modular upgrades and maintenance
- selection of electronic components such as motors, drivers, microcontrollers
- Mathematical models such as forwars and inverse kinematics

- programming the microcontroller, communication, and trajectory planning;

The fact that the manipulator has limited payload is not a drawback. The robot was not designed to lift heavy industrial loads, but rather to work with lightweight tools or execute precise operations (such as moving small objects or demonstrating sorting algorithms). In fact, this lower payload and light weight increases safety, ensuring that programming errors made by students will not result in equipment damage or personal injury.

9.3 Future improvements

Although the robotic arm is fully functional, this is the first version, and it should receive future improvements. The crucial software upgrade is adding a vision system for closed loop control. With a camera and machine learning algorithms, the robot could correct its movements, detect objects, and pick the chosen items.

Another interesting improvement would be constructing a second manipulator for multi robot cooperation and collision avoidance within a shared workspace while executing the desired task.

In addition, some improvements are necessary for the current design. The temporary breadboard circuits should be replaced with a Printed Circuit Board (PCB) to ensure that hardware is not an issue when tackling on a hard task. Another things are cables. Hiding them inside the 3D-printed case would eliminate the risk of tangling and make the whole build look cleaner. Finally, the precision of the Tool Center Point could be improved with the mechanical tolerances of the 3D-printed gear to minimize backlash.

References

- [1] Robotnik. History of robots and robotics | origins of robots, 2021. URL <https://robotnik.eu/history-of-robots-and-robotics/>. Accessed: 2026-01-26.
- [2] scioly. Robot arm, 2025. URL https://scioly.org/wiki/Robot_Arm. Accessed: 2026-06-16.
- [3] FANUC. Fanuc - world market leader in factory automation. <https://www.fanuc.eu/pl-pl/produkt/robot/m-1000ia>. Accessed: 2026-01-30.
- [4] ASTOR. Astor - sklep internetowy. <https://www.astor.com.pl/sklep/robotyzacja/robot-edukacyjny-astorino/astorino-basic-b002.html>, 2025. Accessed: 2026-03-17.
- [5] Chris Annin. Learn robotics with arduino and the ar4 diy robot arm. https://anninrobotics.com/learn-robotics-with-arduino-and-the-ar4-diy-robot-arm/?srsltid=AfmBOoojhTjv1I_OnwyaGcLue_-pt6pgk19J_9YhnjPoUnE2FqKPJyxb/. Accessed: 2026-01-30.
- [6] alldatasheet.pl. Nema17 pdf, 2020. URL <https://www.alldatasheet.com/datasheet-pdf/view/2268109/MOTIONKING/17HS4401S.html>. Accessed: 2026-03-17.
- [7] luna.co. Nema23 pdf, 2020. URL <https://luna.co.in/wp-content/uploads/2019/12/nema23-57HS-2-phase-1.8%C2%B0.pdf>. Accessed: 2026-03-17.
- [8] Markforged. 3d printing settings impacting part strength, 2025. URL <https://markforged.com/resources/learn/design-for-additive-manufacturing-plastics-composites/understanding-3d-printing-strength/3d-printing-settings-impacting-part-strength>. Accessed: 2026-04-02.
- [9] digikey.com. Nema17 pdf, 2020. URL <https://www.digikey.com/en/products/detail/olimex-ltd/SM-17HS4023/21662354>. Accessed: 2026-03-17.
- [10] electronicoscaldas.com. sg995 pdf, 2020. URL https://www.electronicoscaldas.com/datasheet/MG995_Tower-Pro.pdf. Accessed: 2026-03-17.
- [11] STMicroelectronics. Stm32 h755zi-q, 2020. URL <https://www.st.com/en/evaluation-tools/nucleo-h755zi-q.html>. Accessed: 2026-03-17.

- [12] Carlos Ross. Systems and robotics — course resources, 2024. URL <https://biomechatronics.ca/teaching/sr/>. Associate Professor, P.Eng.; Ph.D. (Sorbonne Université, UPMC). Accessed: 2026-01-07.
- [13] circuitist. How to set your stepper driver current: A4988, drv8825, tmc2208, tmc2209, 2022. URL <https://www.circuitist.com/how-to-set-driver-current-a4988-drv8825-tmc2208-tmc2209/>. Accessed: 2026-04-29.